authorization HTTP/TLS/JSON

Interface Design Description

Service ID: *"authorization"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `authorization` service of the ConsumerAuthorization system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation grant . . . 4
2.2 operation revoke . . . 5
2.3 operation lookup . . . 5
2.4 operation verify . . . 6

3 Data Models **8**
3.1 struct AuthorizationGrantRequest . . . 8
3.2 struct AuthorizationLookupRequest . . . 8
3.3 struct AuthorizationPolicy . . . 8
3.4 struct AuthorizationPolicyListResponse . . . 8
3.5 struct AuthorizationPolicyRequest . . . 9
3.6 struct AuthorizationPolicyResponse . . . 9
3.7 struct AuthorizationVerifyRequest . . . 9
3.8 struct ErrorMessage . . . 10
3.9 struct MetadataRequirement . . . 10
3.10 Primitives . . . 10

4 References **11**

5 Revision History **12**
5.1 Amendments . . . 12
5.2 Quality Assurance . . . 12

1 Overview

This document describes the `authorization` service interface of the ConsumerAuthorization system, which provides provider-level authorization policies: a provider declares which consumers may use its service (or event type), and any system can ask whether a consumer is authorized.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `authorization` service interface

This document provides the Interface Design Description IDD to the *authorization – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `authorization` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8445 and base path `/consumerauthorization/authorization`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`grant`	POST	`/grant`
`revoke`	DELETE	`/revoke/{instanceId}`
`lookup`	POST	`/lookup`
`verify`	POST	`/verify`

2 Service Operations

The interfaces of the `authorization` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `authorization` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /consumerauthorization/authorization/grant

Operation: grant
Input: AuthorizationGrantRequest
Output: AuthorizationPolicyResponse

Creates or replaces the provider-level policy of the requesting provider for one target. Returns 201 for a new policy, 200 if an identical one exists.

A successful call returns `201 Created` for a new entry, `200 OK` if an identical entry already existed with the output in the body.

```
POST /consumerauthorization/authorization/grant HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "cloud": "LOCAL",
  "targetType": "SERVICE_DEF",
  "target": "temperatureReading",
  "description": "Displays may read the temperature",
  "defaultPolicy": {
    "policyType": "WHITELIST",
    "policyList": [
      "TemperatureDisplay"
    ],
    "policyMetadataRequirement": {
      "unit": "celsius"
    }
  },
  "scopedPolicies": {
    "read": {
      "policyType": "ALL"
    }
  }
}
```

Listing 1: An grant invocation.

```
{
  "instanceId": "TemperatureSensor|temperatureReading|1.0.0",
  "level": "MGMT",
  "cloud": "LOCAL",
  "provider": "TemperatureSensor",
  "targetType": "SERVICE_DEF",
```

```
  "target": "temperatureReading",
  "description": "Display may read",
  "defaultPolicy": {
    "policyType": "ALL",
    "policyList": [
      "TemperatureDisplay"
    ],
    "policyMetadataRequirement": {
      "unit": "celsius"
    }
  },
  "scopedPolicies": {
    "read": {
      "policyType": "ALL"
    }
  },
  "createdBy": "Sysop",
  "createdAt": "2026-10-22T12:00:00Z"
}
```

Listing 2: An grant response.

2.2 DELETE /consumerauthorization/authorization/revoke/*{instanceId}*

Operation: revoke
Input: String

Removes a provider-level policy of the requester by instance identifier. Returns 200 if removed, 204 if it did not exist.

Path parameter: *instanceId*.

A successful call returns `200 OK` if the entry was removed, `204 No Content` if there was nothing to remove.

```
DELETE /consumerauthorization/authorization/revoke/<instanceId> HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An revoke invocation.

2.3 POST /consumerauthorization/authorization/lookup

Operation: lookup
Input: AuthorizationLookupRequest
Output: AuthorizationPolicyListResponse

Returns the requester's provider-level policies matching the filter.

A successful call returns `200 OK` with the output in the body.

```
POST /consumerauthorization/authorization/lookup HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "instanceIds": [
    "TemperatureSensor|temperatureReading|1.0.0"
  ],
  "cloudIdentifiers": [
    "string"
  ],
  "targetNames": [
```

```
    "string"
  ],
  "targetType": "SERVICE_DEF"
}
```

Listing 4: An lookup invocation.

```
{
  "entries": [
    {
      "instanceId": "TemperatureSensor|temperatureReading|1.0.0",
      "level": "MGMT",
      "cloud": "LOCAL",
      "provider": "TemperatureSensor",
      "targetType": "SERVICE_DEF",
      "target": "temperatureReading",
      "description": "Display may read",
      "defaultPolicy": {
        "policyType": "ALL",
        "policyList": [
          "TemperatureDisplay"
        ],
        "policyMetadataRequirement": {
          "unit": "celsius"
        }
      },
      "scopedPolicies": {
        "read": {
          "policyType": "ALL"
        }
      },
      "createdBy": "Sysop",
      "createdAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 5: An lookup response.

2.4 POST /consumerauthorization/authorization/verify

Operation: verify
Input: AuthorizationVerifyRequest
Output: Boolean

Evaluates whether the consumer may use the target (and operation) of the provider: management-level policies first, provider-level policies if none applies; no applicable policy means denied. Returns a boolean.

A successful call returns `200 OK` with the output in the body.

```
POST /consumerauthorization/authorization/verify HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "provider": "TemperatureSensor",
  "consumer": "TemperatureDisplay",
  "cloud": "LOCAL",
  "targetType": "SERVICE_DEF",
  "target": "temperatureReading",
  "scope": "read"
}
```

Listing 6: An verify invocation.

```
true
```

Listing 7: An verify response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.10, which are used to represent things like hashes, identifiers and texts.

3.1 struct AuthorizationGrantRequest

A provider-level authorization policy that the provider sets for its own service.

Object Field	Value Type	Description
"cloud"	String	Cloud identifier of the consumer cloud the policy applies to; `LOCAL` for the own cloud.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Service definition (or event type) the policy protects.
"description"	String	Free-text description.
"defaultPolicy"	AuthorizationPolicyRequest	Policy that applies to all operations without a scoped policy.
"scopedPolicies"	Object<AuthorizationPolicyRequest>	Policies for individual operations, keyed by operation name.

3.2 struct AuthorizationLookupRequest

Filter for the requester's own provider-level policies.

Object Field	Value Type	Description
"instanceIds"	Array<String>	Policy instance identifiers to match.
"cloudIdentifiers"	Array<String>	Cloud identifiers to match.
"targetNames"	Array<String>	Target names to match.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.

3.3 struct AuthorizationPolicy

One policy rule.

Object Field	Value Type	Description
"policyType"	AuthorizationPolicyType	Policy type: `ALL`, `WHITELIST`, `BLACKLIST` or `SYS_METADATA`.
"policyList"	Array<String>	System names of the white- or blacklist.
"policyMetadata Requirement"	MetadataRequirement	Metadata the consumer system must have (`SYS_METADATA`).

3.4 struct AuthorizationPolicyListResponse

List of authorization policies.

Object Field	Value Type	Description
"entries"	Array<AuthorizationPolicy Response>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.5 struct AuthorizationPolicyRequest

One policy rule.

Object Field	Value Type	Description
"policyType"	String	Policy type: `ALL`, `WHITELIST`, `BLACKLIST` or `SYS_METADATA`.
"policyList"	Array<String>	System names of the white- or blacklist.
"policyMetadata Requirement"	MetadataRequirement	Metadata the consumer system must have (`SYS_METADATA`).

3.6 struct AuthorizationPolicyResponse

A stored authorization policy.

Object Field	Value Type	Description
"instanceId"	String	Policy instance identifier `<MGMT or PR>|<cloud>|<provider>|<targetType>|<target>`.
"level"	AuthorizationLevel	Policy level: `MGMT` (set by an operator) or `PROVIDER` (set by the provider itself).
"cloud"	String	Cloud identifier of the consumer cloud the policy applies to; `LOCAL` for the own cloud.
"provider"	String	Provider system the policy protects.
"targetType"	AuthorizationTargetType	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"description"	String	Free-text description.
"defaultPolicy"	AuthorizationPolicy	Policy that applies to all operations without a scoped policy.
"scopedPolicies"	Object<AuthorizationPolicy>	Policies for individual operations, keyed by operation name.
"createdBy"	String	Name of the system that created the record.
"createdAt"	DateTime	UTC time of creation.

3.7 struct AuthorizationVerifyRequest

One authorization question: may the consumer use the target (operation) of the provider?

Object Field	Value Type	Description
"provider"	String	Provider system; for `verify` defaults to the requester.
"consumer"	String	Consumer system; for `verify` defaults to the requester.
"cloud"	String	Cloud of the consumer, `LOCAL` by default.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"scope"	String	Operation to check; empty checks the default policy.

3.8 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.9 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.10 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
AuthorizationLevel	One of `MGMT`, `PROVIDER` (JSON String).
AuthorizationPolicyType	One of `ALL`, `WHITELIST`, `BLACKLIST`, `SYS_METADATA` (JSON String).
AuthorizationTargetType	One of `SERVICE_DEF`, `EVENT_TYPE` (JSON String).
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “authorization – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorization-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	